

Towards five-state minimal-time firing squads: a pumping lemma, a speed-1/3 barrier, and certified bounds

Anonymous

Affiliation withheld for review

Abstract

Whether the firing squad synchronization problem admits a minimal-time solution with five states has been open since Mazoyer’s six-state solution of 1987. We develop general tools for minimal-time solutions and measure how far they reach on this question. (i) All lines are encoded by one half-line diagram and one reflected triangle per length, and every triangle is a function of two anti-diagonals of the half-line. (ii) We prove a pumping lemma: in every minimal-time solution, with any number of states, the inputs of the reflected triangles of two lengths $n < n'$ differ on an explicit cone of about $n/2$ cells. The lemma constrains the half-line alone and yields a speed-1/3 barrier: the non-periodic part of the half-line always reaches the line $i = t/3$, which explains the speed-1/3 signal of the classical constructions. (iii) A propositional encoding of the bounded problem, validated against Mazoyer’s solution extracted from its Coq formalization, gives an LRAT-certified proof, checked in six seconds, that no four-state minimal-time solution exists, even without the left quiescence condition. (iv) For five states we exhibit an explicit rule that synchronizes every line of length at most 12 in minimal time, whose half-line obeys the pumping constraints up to length 40 and never fires up to anti-diagonal 78, and whose last configuration before firing is uniform. Hence any refutation of five-state solutions must involve lines of length at least 13. Random-probe estimates of the natural look-ahead search tree give about 27 nodes for four states but 10^{11} to 10^{19} for five, which quantifies why the question remains open.

Keywords: firing squad synchronization problem, cellular automata, state complexity, minimal time, SAT solving, proof certificates

1. Introduction

The firing squad synchronization problem (FSSP), posed by Myhill in 1957 and popularised by Moore [1], asks for a finite automaton that, placed in every cell of a line of arbitrary length n and started with all cells quiescent except a *general* at one end, drives all cells into a common *firing* state for the first time at the same instant. Goto [2] and Waksman [3] showed that this is possible in time $2n - 2$, which is optimal [1]. Since then the minimum number of states of a *minimal-time* solution has been a benchmark question in cellular automata theory [4, 5]: Waksman used 16 states, Balzer 8 [6], Gerken 7 [7] and Mazoyer 6 [8]. On the other side, Balzer [6] reported, and Sanders [9] confirmed with a corrected exhaustive search, that no 4-state minimal-time solution exists. Whether five states suffice has remained open since Mazoyer’s construction; it is regarded as the central open question on state complexity of the FSSP [5, 10]. Many six-state minimal-time solutions are known today [11, 12], and solutions with at most five states are known for restricted families of lengths, for instance $n = 2^k$, some of them in non-minimal time [13, 14, 15], but no five-state minimal-time solution has been found, nor has its existence been refuted.

This paper. We do not settle the five-state question. We develop tools that apply to minimal-time solutions with any number of states, use them to give a certified proof of the four-state lower bound that is produced and checked within a minute, and measure precisely how far the same tools reach for five states.

- (i) *Agreement and reflection* (Section 2). The line of length n coincides with the half-line below the anti-diagonal $t + i = 2n - 2$; hence all lines are described by a single half-line diagram C_∞ plus one *reflected triangle* R_n per length, and R_n is a function of two anti-diagonals of C_∞ only (Lemmas 2.2 and 3.1).
- (ii) *A pumping lemma and a speed-1/3 barrier* (Section 3). In every minimal-time solution the input words of the reflected triangles of any two lengths $4 \leq n < n'$ differ on an explicit cone of about $n/2$ cells (Lemma 3.3). This is a constraint on the half-line alone, valid for *all* pairs of lengths. It implies that the non-periodic part of the half-line always reaches the line $i = t/3$ (Theorem 3.4), an a priori explanation of the speed-1/3 signal present in the classical constructions.
- (iii) *A validated encoding and a certified four-state bound* (Sections 4 and 5). We encode the bounded problem, together with the consequences

above, into propositional logic, validate the encoding against Mazoyer’s solution extracted mechanically from Duprat’s Coq formalization [16], and obtain an LRAT certificate, checked in 6 seconds, that no four-state minimal-time solution exists (Theorem 5.1); the statement does not need the left quiescence condition $\delta(*, L, L) = L$.

- (iv) *The five-state frontier* (Section 6). We exhibit an explicit five-state rule that synchronizes every line of length at most 12 in minimal time, satisfies all pumping constraints up to length 40, never fires on the half-line up to anti-diagonal 78, and even ends with a uniform configuration (Theorem 6.1); so every refutation of five-state solutions must use lines of length at least 13. We show that all the difficulty lies in the half-line: once its transitions are fixed, completing the rule is decided in a fraction of a second, whereas random-probe estimates of the natural look-ahead search tree grow from about 27 nodes for four states to 10^{11} – 10^{19} for five.

Related work. Surveys of the FSSP are [4, 5]; a classical textbook exposition is [17]. Mazoyer [18] studied the structure of optimal solutions and the information flow they require; Settle and Simon [19] obtained small non-minimal-time solutions. Beyond Balzer and Sanders, lower bounds on the number of states are known for variants: no three-state solution and no four-state symmetric minimal-time solution exist for rings [20]; for further restricted classes see [10]. Computer searches have been used to find new solutions [11] and to generate families of solutions by local simulations [12]. SAT solvers have been applied to many extremal problems in combinatorics with certified unsatisfiability proofs [21, 22]; to our knowledge this is the first SAT-based treatment of the FSSP state-complexity question.

2. Preliminaries

2.1. The problem

Let S be a finite set of *states* containing three distinguished elements: the *quiescent* state L , the *general* G and the *firing* state F ; these are pairwise distinct. We write $W := S \setminus \{F\}$ for the set of *working* states and use a symbol $*$ $\notin S$ for the border. A *rule* is a map

$$\delta : (W \cup \{*\}) \times W \times (W \cup \{*\}) \longrightarrow S$$

satisfying the quiescence conditions

$$\delta(L, L, L) = \delta(L, L, *) = L. \tag{1}$$

(Transitions out of F and transitions whose neighbourhood contains F play no role below and are left unspecified; we do *not* require $\delta(*, L, L) = L$, which only strengthens the non-existence statements.)

For $n \geq 1$ the *line of length n* is the space-time diagram $C_n : \mathbb{N} \times \{1, \dots, n\} \rightarrow S$ defined by $C_n(0, 1) = G$, $C_n(0, i) = L$ for $2 \leq i \leq n$, and

$$C_n(t+1, i) = \delta(C_n(t, i-1), C_n(t, i), C_n(t, i+1)),$$

with the convention $C_n(t, 0) = C_n(t, n+1) = *$; the diagram is only considered up to the first time at which some cell is in state F. The rule *synchronizes the line of length n at time T* if $C_n(T, i) = F$ for all $1 \leq i \leq n$ and $C_n(t, i) \neq F$ for all $t < T$ and all i .

It is classical [1, 3, 6] that a rule which synchronizes *every* line cannot synchronize the line of length $n \geq 2$ before time $2n - 2$ (see Remark 2.4). A *minimal-time solution* is a rule that synchronizes the line of length n at time exactly $2n - 2$ for every $n \geq 2$. (For $n = 1$ one usually asks $\delta(*, G, *) = F$; this transition is used by no other line and is irrelevant here.) A *k -state solution* is one with $|S| = k$.

More generally, for a set $\mathcal{N} \subseteq \{2, 3, \dots\}$ of lengths, a rule is *$\mathcal{N}$ -minimal* if it synchronizes the line of length n at time $2n - 2$ for every $n \in \mathcal{N}$. A minimal-time solution is $\mathcal{N}$ -minimal for every $\mathcal{N}$. Since there are finitely many rules with k states, k -state minimal-time solutions do not exist if and only if there is a finite N such that no k -state rule is $\{2, \dots, N\}$ -minimal; a non-existence proof may therefore consist of a finite family of necessary conditions that no k -state rule satisfies. This is how we prove the four-state bound (Theorem 5.1); for five states we study how large such a family must be.

2.2. The half-line and the agreement lemma

Let $C_\infty : \mathbb{N} \times \{1, 2, \dots\} \rightarrow S$ be the diagram of the half-line: $C_\infty(0, 1) = G$, $C_\infty(0, i) = L$ for $i \geq 2$, and the same recurrence with a border on the left only.

Lemma 2.1 (light cone). *For every rule, every n and every (t, i) with $t < i - 1$ we have $C_n(t, i) = C_\infty(t, i) = L$.*

Proof. By induction on t . For $t = 0$ this is the initial condition. Let $t \geq 1$ and $t < i - 1$. The three cells $(t - 1, i - 1)$, $(t - 1, i)$, $(t - 1, i + 1)$ satisfy $t - 1 < (i - 1) - 1$, hence are in state L by induction, except that the right neighbour of cell n of C_n is the border. By (1) the new state is L in both cases. $\square$

Lemma 2.2 (agreement). *For every rule and every $n \geq 2$: $C_n(t, i) = C_\infty(t, i)$ whenever $1 \leq i \leq n$ and $t + i \leq 2n - 2$.*

Proof. Induction on t ; the case $t = 0$ is clear. Let $t \geq 1$ and $t + i \leq 2n - 2$. If $i = n$ then $t \leq n - 2 < i - 1$ and both sides equal $\mathbf{L}$ by Lemma 2.1. If $i < n$, the three neighbours $(t - 1, j)$, $j \in \{i - 1, i, i + 1\}$, satisfy $(t - 1) + j \leq t + i \leq 2n - 2$ and $j \leq n$, so they agree by induction (for $j = 0$ both are the border). Hence the new states agree. $\square$

Thus the right border of the line of length n is first “felt” at the cell $(n - 1, n)$, and its influence is confined to the *reflected triangle*

$$R_n := \{(t, i) : 1 \leq i \leq n, t + i \geq 2n - 1, t \leq 2n - 2\}.$$

Lemma 2.3 (return chain). *Let δ synchronize the lines of lengths n and $n + 1$ at times $2n - 2$ and $2n$ respectively. Then*

$$C_n(2n - 1 - i, i) \neq C_\infty(2n - 1 - i, i) \quad (1 \leq i \leq n).$$

*In particular the front $s_i := C_\infty(i - 1, i)$ satisfies $s_{n-1} \neq \mathbf{L}$, and $\delta(s_{n-1}, \mathbf{L}, *) \neq \delta(s_{n-1}, \mathbf{L}, \mathbf{L})$.*

Proof. For $i = 1$: $C_n(2n - 2, 1) = \mathbf{F}$ while, by Lemma 2.2 applied to the line of length $n + 1$ (note $(2n - 2) + 1 \leq 2(n + 1) - 2$), $C_\infty(2n - 2, 1) = C_{n+1}(2n - 2, 1) \neq \mathbf{F}$ because $2n - 2 < 2n$. Suppose the claim holds for some $i < n$. The cell $(2n - 1 - i, i)$ is computed from the cells $(2n - 2 - i, j)$, $j = i - 1, i, i + 1$, whose coordinate sums are $2n - 3$, $2n - 2$ and $2n - 1$. By Lemma 2.2 the first two are the same in C_n and C_∞ (for $i = 1$ the left neighbour is the border in both diagrams). Since the results differ, the right neighbours differ: $C_n(2n - 2 - i, i + 1) \neq C_\infty(2n - 2 - i, i + 1)$, which is the claim for $i + 1$. For $i = n$ we get $C_n(n - 1, n) = \delta(s_{n-1}, \mathbf{L}, *) \neq C_\infty(n - 1, n) = \delta(s_{n-1}, \mathbf{L}, \mathbf{L}) = s_n$, which is impossible if $s_{n-1} = \mathbf{L}$ by (1). $\square$

Remark 2.4. Suppose a rule synchronizes every line, the line of length n at some time $T \leq 2n - 3$. By Lemma 2.2, $C_\infty(T, 1) = C_n(T, 1) = \mathbf{F}$, hence $C_m(T, 1) = \mathbf{F}$ for every $m \geq n$. For $m > T + 1$ the cell m is still quiescent at time T by Lemma 2.1, so the line of length m has a firing cell at time T without firing entirely: a contradiction.

Applying Lemma 2.3 to all n shows that in a minimal-time solution every cell i leaves $\mathbf{L}$ exactly at time $i - 1$ (the front moves at speed one) and, since $C_\infty(t, i) = C_n(t, i)$ for n large,

$$C_\infty(t, i) \neq \mathbf{F} \quad \text{for all } t \geq 0, i \geq 1. \quad (2)$$

3. Reflected triangles and a pumping lemma

3.1. Relative coordinates

Fix $n \geq 2$. For a cell (t, i) of the line of length n put $\tau := t - (n - 1)$ (time elapsed since the front reached the right end) and $\kappa := n - i$ (distance to the right end). Then

$$R_n = \{(\tau, \kappa) : 0 \leq \kappa \leq \tau \leq n - 1\},$$

the right border sits at $\kappa = -1$ and the left border at $\kappa = n$. A cell $(\tau, \kappa) \in R_n$ is computed from $(\tau - 1, \kappa + 1), (\tau - 1, \kappa), (\tau - 1, \kappa - 1)$; each of these is either in R_n , or a border, or satisfies $\tau' - \kappa' \in \{-1, -2\}$, i.e. lies on one of the anti-diagonals $t + i = 2n - 2, t + i = 2n - 3$, where C_n coincides with C_∞ by Lemma 2.2. We call

$$J_n(\kappa) := (C_\infty(n - 3 + \kappa, n - \kappa), C_\infty(n - 2 + \kappa, n - \kappa)), \quad 0 \leq \kappa \leq n - 1,$$

the *input word* of R_n ; its two components are the *lower* ($t + i = 2n - 3$) and *upper* ($t + i = 2n - 2$) input cells.

Lemma 3.1 (determinacy). *There is a single partial map Ψ , depending on δ only, such that for every $n \geq 2$ and every $(\tau, \kappa) \in R_n$, $C_n(n - 1 + \tau, n - \kappa)$ is obtained by evaluating Ψ at (τ, κ) on the input word J_n with borders at $\kappa = -1$ and $\kappa = n$. In particular the value only depends on the input cells and borders in the backward dependency cone of (τ, κ) .*

Proof. The recurrence in relative coordinates does not mention n ; only the position of the left border does. $\square$

For $(\tau, \kappa) \in R_n$ let $\text{cone}_n(\tau, \kappa)$ be the set of input cells, i.e. of pairs consisting of a position κ' and one of the two anti-diagonals, that are reached from (τ, κ) by repeatedly passing to the three predecessors, stopping at input cells and borders; say that the cone is *right-anchored* if it does not reach the left border $\kappa' = n$.

Lemma 3.2. *For $n \geq 4$ the cone of the right end at the firing time, $\text{cone}_n(n - 1, 0)$, is right-anchored and consists of the upper input cell at $\kappa' = 0$, both input cells at $1 \leq \kappa' \leq \lfloor n/2 \rfloor$, and, when n is odd, the lower input cell at $\kappa' = (n + 1)/2$.*

Proof. A predecessor step changes τ by -1 and κ by at most one, and input cells satisfy $\tau' = \kappa' - 1$ or $\kappa' - 2$. Starting from $(n - 1, 0)$, a path that reaches an input cell at position κ' after s steps satisfies $\kappa' \leq s$ and $n - 1 - s = \kappa' - 1$

or $\kappa' - 2$; hence $2\kappa' \leq n + 1$ (upper cells: $2\kappa' \leq n$). Conversely, the upper cell at a position κ' with $2\kappa' \leq n$ is reached by moving left κ' times and then staying put, and the lower cell at a position $\kappa' \geq 1$ with $2\kappa' \leq n + 1$ by moving left $\kappa' - 1$ times, staying put down to the cell $(\kappa' - 1, \kappa' - 1)$ and moving left once more; all intermediate cells satisfy $\tau \geq \kappa$ and lie in R_n . The lower cell at $\kappa' = 0$ is a predecessor of no cell of R_n . The largest position reached is $\lceil n/2 \rceil \leq n - 1$, and a path can reach $\kappa = n$ only after n left moves, i.e. at a negative time, so the left border is never reached; the right border only contributes the constant symbol $*$. (The same set is also computed mechanically in the accompanying code, file `pumping.py`.) $\square$

Lemma 3.3 (pumping). *Let δ be a minimal-time solution and $4 \leq n < n'$. Then J_n and $J_{n'}$ differ on $\text{cone}_n(n - 1, 0)$.*

Proof. Suppose they agree there. By Lemmas 3.1 and 3.2 the relative cell $(n - 1, 0)$ of $R_{n'}$ (which belongs to $R_{n'}$ since $n - 1 \leq n' - 1$, and whose cone is the same set of input positions, far from the left border $\kappa = n' > n$) has the same value as the relative cell $(n - 1, 0)$ of R_n , namely $C_n(2n - 2, n) = F$. Thus $C_{n'}(n' + n - 2, n') = F$ although $n' + n - 2 < 2n' - 2$, contradicting minimality for n' . $\square$

The point of Lemma 3.3 is that it constrains the *half-line diagram only*: no reflected triangle has to be computed. It is the source of the constraint family $\text{PUMP}(n, n')$ used in Section 4. As a sanity check, we verified mechanically that Mazoyer's six-state solution satisfies $\text{PUMP}(n, n')$ for all $4 \leq n < n' \leq 120$.

3.2. Consequence: a speed-1/3 barrier

Write the half-line diagram in the frame of the front: $\Phi_t(j) := C_\infty(t, t + 1 - j)$ for $0 \leq j \leq t$ (the cell at *depth* j behind the front at time t). By the recurrence,

$$\Phi_{t+1}(j) = \delta(\Phi_t(j), \Phi_t(j - 1), \Phi_t(j - 2)) \quad (0 \leq j \leq t), \quad (3)$$

with $\Phi_t(-1) = \Phi_t(-2) = L$: in this frame the half-line is a *one-sided* automaton in which information only travels to larger depths, at most two depths per step. In particular every depth-row $t \mapsto \Phi_t(j)$ is eventually periodic. In these coordinates $J_n(\kappa) = (\Phi_{n-3+\kappa}(2\kappa - 2), \Phi_{n-2+\kappa}(2\kappa - 1))$.

Theorem 3.4 (speed-1/3 barrier). *Let δ be a minimal-time solution. Every depth-row $t \mapsto \Phi_t(j)$ is eventually periodic; let $T(j)$ be the time from which it is periodic. Then for every $n \geq 4$ there is a depth $j \leq n$ with $T(j) > n + \frac{j}{2} - \frac{5}{2}$.*

Consequently $\limsup_{j \rightarrow \infty} T(j)/j \geq 3/2$: along infinitely many depth-rows the transient part extends at least to the line $i = t/3$ of the original coordinates.

Proof. Row 0 satisfies $\Phi_{t+1}(0) = \delta(\Phi_t(0), \mathbf{L}, \mathbf{L})$ and is eventually periodic; by (3), row j is a finite-state system driven by the eventually periodic rows $j-1$, $j-2$, hence eventually periodic, with some period P_j . Fix $n \geq 4$ and let P be a common multiple of $P_0, \dots, P_{n+1}$. If every input cell of $\text{cone}_n(n-1, 0)$ were read in the periodic regime of its row, i.e. $n-3+\kappa \geq T(2\kappa-2)$ and $n-2+\kappa \geq T(2\kappa-1)$ for all cone positions κ , then $J_{n+P} = J_n$ on the cone, contradicting Lemma 3.3. Cone positions satisfy $\kappa \leq \lceil n/2 \rceil$, so the violated inequality concerns a depth $j \in \{2\kappa-2, 2\kappa-1\}$ with $j \leq n$ and $T(j) > n-3+\kappa \geq n + \frac{j}{2} - \frac{5}{2}$. If $T(j) \leq \alpha j + \beta$ for all j with $\alpha < 3/2$, then for n large and every $j \leq n$ we would have $(\alpha - \frac{1}{2})j \leq (\alpha - \frac{1}{2})n < n - \beta - \frac{5}{2}$, i.e. $T(j) \leq n + \frac{j}{2} - \frac{5}{2}$, a contradiction. Finally a cell of depth j at time $T(j)$ sits at position $T(j) + 1 - j$, which is at least about $T(j)/3$ when $T(j) \geq 3j/2$. $\square$

Theorem 3.4 explains a posteriori why every minimal-time construction contains structure reaching the line $i = t/3$: the speed-1/3 signal emitted by the general in the constructions of Waksman and Balzer [3, 6] lies exactly on it, and in Mazoyer’s solution the periodic zone behind the front is bounded by a line of speed 1/2: we checked that for $1 \leq j < 700$ the depth-row j is 3-periodic from time $T(j) = 2j + 2$ when j is even or $j = 1$, and from $T(j) = 2j + 1$ when $j \geq 3$ is odd. The crossing of this line with the anti-diagonal $t + i = 2n - 2$ is exactly the far end of $\text{cone}_n(n-1, 0)$. The theorem also explains the failure of many of the “lucky” partial rules found by the solvers (Section 6): their half-line diagrams are periodic behind the front far beyond the line $i = t/3$.

4. Propositional encoding

Fix k (the number of states), write $W = \{\mathbf{L}, \mathbf{G}, a_1, \dots, a_{k-3}\}$ and let $\mathcal{N}$ be a finite set of lengths. We build a CNF formula $\text{MT}_k(\mathcal{N}; M, N')$ that is satisfiable whenever a k -state minimal-time solution exists. All constraint families below are either the definition of $\mathcal{N}$ -minimality or consequences of Lemmas 2.2, 2.3, 3.1 and 3.3 and (2); none relies on unproved assumptions about the shape of solutions.

Variables. For every neighbourhood $e = (x, y, z) \in (W \cup \{*\}) \times W \times (W \cup \{*\})$ other than $(*, y, *)$, one-hot variables $T_{e,d}$, $d \in S$, encode $\delta(e) = d$; the two quiescence transitions (1) are fixed by unit clauses. For every cell whose

state is not a constant, one-hot variables $X_{c,s}$, $s \in W$, encode its state (all cells whose state is used before the firing time are in W).

Transition clauses. For a cell c with neighbours c_1, c_2, c_3 (cells, constants L, G , or the border) and for all (x, y, z) in the respective domains and $d \in W$ we add

$$\begin{aligned} &\neg X_{c_1,x} \vee \neg X_{c_2,y} \vee \neg X_{c_3,z} \vee \neg T_{(x,y,z),d} \vee X_{c,d}, \\ &\neg X_{c_1,x} \vee \neg X_{c_2,y} \vee \neg X_{c_3,z} \vee \neg T_{(x,y,z),F} \end{aligned}$$

(the second clause forbids firing). For a cell that must fire we add instead $\neg X_{c_1,x} \vee \neg X_{c_2,y} \vee \neg X_{c_3,z} \vee T_{(x,y,z),F}$. Literals of constant neighbours are omitted.

Half-line. The cells (t, i) of C_∞ with $1 \leq i \leq t + 1$ and $t + i \leq M$ receive variables and transition clauses; cells with $i > t + 1$ are the constant L (Lemma 2.1). Every such cell is forbidden to fire, by (2).

Lines. For every $n \in \mathcal{N}$ (with $2n - 2 \leq M$) the cells of R_n with $t \leq 2n - 3$ receive variables; their neighbours outside R_n are the corresponding half-line cells (Lemma 2.2). All of them are forbidden to fire and the n cells at time $2n - 2$ are required to fire.

Return chains. For $n \in \mathcal{N}$ with $2n - 1 \leq M$ we add the implied constraints $C_n(2n - 1 - i, i) \neq C_\infty(2n - 1 - i, i)$ of Lemma 2.3 (their derivation only uses (2) at the cell $(2n - 2, 1)$, which is part of the encoding).

Pumping. For all $4 \leq n < n' \leq N'$ (with $2N' - 2 \leq M$) we add one clause stating that J_n and $J_{n'}$ differ on $\text{cone}_n(n - 1, 0)$, introducing an auxiliary “difference” variable for every pair of non-constant input cells (Lemma 3.3).

Determinacy links. For $n < n'$ in $\mathcal{N}$ and $p \geq 0$ an auxiliary variable $E_{n,n'}(p)$ is made equivalent to “ J_n and $J_{n'}$ coincide on the positions $0, \dots, p$ ”. For every relative cell (τ, κ) of R_n whose dependency cone is right-anchored and contained in the positions $0, \dots, p$ we add $E_{n,n'}(p) \Rightarrow (R_n(\tau, \kappa) = R_{n'}(\tau, \kappa))$, and, for the firing cells $\tau = n - 1$, the clause $\neg E_{n,n'}(p)$ (Lemmas 3.1 and 3.3). These clauses are implied by the others but allow the solver to transfer information between lines.

Symmetry breaking. Permuting the auxiliary states maps solutions to solutions. We require that, in the time-major enumeration of the encoded half-line cells, the first occurrence of a_j precedes the first occurrence of a_{j+1} ; every solution can be renamed to satisfy this. For $k = 5$ this halves the search space.

Soundness checks. Besides the proofs above, every constraint family was tested against an independent ground truth: Mazoyer’s six-state solution, extracted mechanically from Duprat’s Coq formalization [16], satisfies the formula $\text{MT}_6(\mathcal{N}; 78, 40)$ for $\mathcal{N} = \{3, \dots, 12\}$ with return chains, pumping and links (the formalization does not cover $n = 2$), and it satisfies $\text{MT}_6(\{3, \dots, 14\}; 26)$ with symmetry breaking under exactly one of the $3!$ renamings of its auxiliary states, as it must. Conversely, every model found by the solvers was decoded and re-simulated by an independent brute-force checker (`fsspcheck.c`), which confirmed minimal-time synchronization for all encoded lengths.

5. Four states: a certified proof

All formulas are produced by the generator `gencnf.py`; models are decoded into rule tables and re-simulated by the independent brute-force checker `fsspcheck.c`. Solvers: Kissat 4.0.4 [23] and CaDiCaL 3.0.1 [24] (or CaDiCaL 1.9.5 through PySAT [25] for incremental runs). Times are single-core wall-clock times on a cloud virtual CPU.

Theorem 5.1. *No rule with four states $\{\mathbf{L}, \mathbf{G}, a, \mathbf{F}\}$ satisfying (1) synchronizes all lines of lengths $2, \dots, 9$ in minimal time. In particular there is no four-state minimal-time solution, even without the condition $\delta(*, \mathbf{L}, \mathbf{L}) = \mathbf{L}$.*

Proof. By Lemmas 2.2 and 2.3 and (2) every such rule yields a model of $\text{MT}_4(\{2, \dots, 9\}; 16)$ (for $k = 4$ there is a single auxiliary state and no symmetry breaking). CaDiCaL proves this formula unsatisfiable in 51 s and emits an LRAT proof of 177 MB (41 MB compressed), which `lrat-check` of the `drat-trim` suite [26] verifies in 5.7 s. The SHA-256 digests of the formula and of the proof are listed in Appendix B. $\square$

The bound 9 is sharp for the plain formula: $\text{MT}_4(\{2, \dots, 8\}; 14)$ is satisfiable, and an independent depth-first enumeration (program `fsspdfs.c`, which fills the table lazily in the order in which transitions are needed) finds the numbers of partial rules listed in Table 1, counted on the transitions they use; it reaches zero exactly at the lengths $2, \dots, 9$, in agreement with the solver. With the half-line extended to anti-diagonal 78 the lengths $2, \dots, 8$ already suffice. The pumping constraints alone are weak for four states: together with the half-line up to anti-diagonal 78, the lengths $2, \dots, 7$ remain satisfiable.

largest length N	5	6	7	8	9
partial rules synchronizing $2, \dots, N$	828,670	64,299	536	27	0

Table 1: Four states: number of partial rules (restricted to the transitions they use) that synchronize every line of length $2, \dots, N$ in minimal time, by exhaustive enumeration ($1.09 \cdot 10^8$ search nodes).

instance ($k = 5$)	result	time
$\text{MT}_5(\{2, \dots, 10\}; 18)$	SAT, fails at $n = 11$	10.8 s
$\text{MT}_5(\{2, \dots, 12\}; 22)$	SAT, fails at $n = 13$	≈ 20 min
$\text{MT}_5(\{2, \dots, 9, 16\}; 30)$	SAT, fails at $n = 10$	23.6 s
$\text{MT}_5(\{2, \dots, 9, 20\}; 38)$	SAT, fails at $n = 10$	82.4 s
$\text{MT}_5(\{2, \dots, 8\}; 78, 40)$	SAT, fails at $n = 9$	68.7 s
$\text{MT}_5(\{2, \dots, 10\}; 78, 40)$	SAT, fails at $n = 11$	755.6 s
$\text{MT}_5(\{2, \dots, 12\}; 78, 40) + \text{links} + \text{uniform pre-firing } A^n$	SAT, fails at $n = 13$	1930 s

Table 2: Five-state instances. $\text{MT}_k(\mathcal{N}; M, N')$: lengths $\mathcal{N}$, half-line up to anti-diagonal M , pumping pairs up to N' .

6. Five states: the frontier

6.1. Lucky rules

For five states the table has 94 free transitions with five values each, against 45 with four values for four states: the search space grows by a factor of about 10^{38} . Table 2 lists representative instances. All of them are satisfiable, and in each case the decoded rule fails for some length that was not encoded; we call such rules *lucky*. Two kinds occur. Rules of the first kind have a *periodic* half-line behind the front (Figure 1(b)); they can even synchronize an isolated long line such as $n = 20$, but they violate the pumping lemma massively (155 of the pairs $4 \leq n < n' \leq 40$ for the rule shown). Rules of the second kind have a chaotic half-line that eventually fires. Once the half-line is extended and the pumping constraints are imposed, the solver is forced to produce half-lines with a genuine slow boundary: in every model we inspected a region delimited by a line of slope $1/3$ appears (Figure 1(c)), exactly as predicted by Theorem 3.4.

6.2. A five-state rule for all lengths up to twelve

Theorem 6.1. *The five-state rule δ_{12} of Table A.3 has the following properties.*

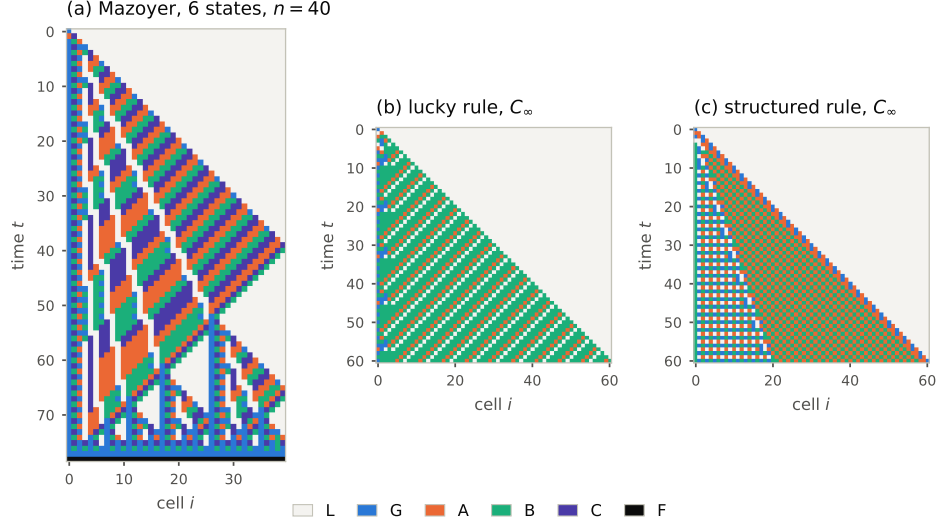


Figure 1: Space-time diagrams, time downwards. (a) Mazoyer’s six-state solution on $n = 40$ cells; the last row is the firing row. (b) Half-line of a five-state rule that synchronizes the lengths $2, \dots, 9$ and 20 but violates the pumping lemma: its background is periodic. (c) Half-line of a five-state rule found under the pumping constraints: the boundary between the checkered region and the diagonal background has slope $1/3$.

- (a) *It synchronizes the line of length n at time $2n - 2$ for every $2 \leq n \leq 12$, and one step before firing all cells are in state A.*
- (b) *Its half-line diagram contains no firing cell (t, i) with $t + i \leq 78$.*
- (c) *It satisfies $\text{PUMP}(n, n')$ for all $4 \leq n < n' \leq 40$.*

It does not synchronize the line of length 13: a cell fires at time 23.

Proof. Each property is a finite computation on the explicit table, carried out by direct simulation with the checker and the independent script `pumping.py` (Appendix B). $\square$

Consequently, the necessary conditions used in this paper, restricted to lengths at most 12 (even together with the pumping constraints up to 40 and the half-line up to anti-diagonal 78), are satisfiable: a refutation of five-state minimal-time solutions along these lines must involve lines of length at least 13, and the requirement “uniform configuration before firing”, satisfied by Mazoyer’s solution, does not help below that length.

6.3. Where the difficulty lies

The half-line determines everything. Take any of the rules found above, fix the transitions it uses on the half-line up to anti-diagonal 78 (between 39 and 50 transitions) and ask whether the remaining transitions can be completed so that all lengths $2, \dots, N$ synchronize. For the five-state germs we tried, Kissat decides this in less than one second: for the rule of Table 2 with lengths $2, \dots, 10$ the completion exists for $N = 8$ and not for $N = 12$; the other germs fail already for $N = 8$. As a sanity check on a genuine solution, fixing the 58 transitions that Mazoyer’s rule uses on its half-line up to anti-diagonal 78 leaves a completion problem for the lengths $3, \dots, 12$ that Kissat solves in 4s. Conversely, the unsatisfiable core returned for such a germ contains 38 of its 39 transitions, so that learning from the completion problem (a logic-based Benders scheme we implemented) does not prune the half-line search effectively.

Size of the search. We estimated the size of the natural look-ahead search tree—half-line transitions fixed in order of first use, anti-diagonal by anti-diagonal, a node being discarded when the full formula with the partial table as assumptions is refuted within 3000 conflicts— with Knuth’s random-probe estimator. For four states (MT_4 with lengths $2, \dots, 9$) the mean over 30 probes is 27 nodes. For five states ($\text{MT}_5(\{2, \dots, 12\}; 78, 40)$ with links) three probes gave $4.2 \cdot 10^{11}$, $3.1 \cdot 10^{17}$ and $8.3 \cdot 10^{18}$. The estimator has a large variance, but the gap of at least ten orders of magnitude is robust and explains why the unrestricted five-state formulas with $N \geq 12$ that we started (up to $N = 18$, several CPU-hours each) did not terminate. Clause learning compresses such trees considerably, and cube-and-conquer [27] parallelizes them almost linearly, but a complete five-state search along these lines is a computation of a much larger scale than the ones reported here.

7. Discussion

What the pumping lemma buys. Lemma 3.3 turns an infinite family of requirements on the reflected triangles into constraints on the half-line diagram only. It is satisfied by Mazoyer’s solution, as it must be, and it eliminates, at negligible cost, the whole class of rules whose half-line becomes periodic behind the front too early. Theorem 3.4 shows that the constraint has a transparent geometric meaning: in every minimal-time solution the non-periodic part of the half-line reaches the line $i = t/3$. For Mazoyer’s solution every depth-row is 3-periodic from time $2j + 1$ or $2j + 2$, i.e. the periodic zone

behind the front is bounded by a signal of speed $1/2$, comfortably within the bound.

Why lucky rules are hard to kill. Small sets of necessary conditions are typically met by rules that are not solutions; the question is how many lengths are needed before all such rules disappear (if no solution exists, some finite set of lengths suffices, since there are finitely many rules). For four states the answer is 9 (or 8 with a long half-line). For five states the rule space is larger by roughly 38 orders of magnitude (5^{94} versus 4^{45} transition tables), and our experiments show partial rules for all lengths up to at least 12, and for any single additional length we tried. Under the pumping constraints the surviving rules are no longer random: their half-lines exhibit a speed- $1/3$ boundary, and the remaining freedom lies in the reflected triangles.

The mirrored construction. The cone of Lemma 3.2 ends where the anti-diagonal $t + i = 2n - 2$ meets the line $i = t/3$, and the information that makes the right end fire exactly at time $2n - 2$ must travel from there to the right end at maximal speed. Consequently the right half of every reflected triangle has to perform a minimal-time synchronization that is *mirrored* and runs on the background left behind the front instead of on quiescent cells. A five-state solution would have to realize both constructions—the direct one on the quiescent background and the mirrored one on the front’s background—with the same four working states. We regard this tension as the structural reason why five states are hard, and we believe that making it quantitative is the most promising route to a conceptual proof.

Data availability

All programs, formula generators, rule tables and experiment logs are available in the accompanying repository (directory `fssp5/`). The LRAT certificate of Theorem 5.1 (177 MB) is not stored; it is regenerated deterministically in under a minute by the commands of Appendix B, and its SHA-256 digest is listed there.

References

- [1] E. F. Moore, The firing squad synchronization problem, in: E. F. Moore (Ed.), *Sequential Machines: Selected Papers*, Addison-Wesley, Reading, MA, 1964, pp. 213–214.

- [2] E. Goto, A minimal time solution of the firing squad problem, Dittoed course notes for Applied Mathematics 298, Harvard University, pp. 52–59 (1962).
- [3] A. Waksman, An optimum solution to the firing squad synchronization problem, *Information and Control* 9 (1) (1966) 66–78.
- [4] J. Mazoyer, An overview of the firing squad synchronization problem, in: C. Choffrut (Ed.), *Automata Networks*, Vol. 316 of *Lecture Notes in Computer Science*, Springer, 1988, pp. 82–94.
- [5] H. Umeo, Firing squad synchronization problem in cellular automata, in: R. A. Meyers (Ed.), *Encyclopedia of Complexity and Systems Science*, Springer, 2009, pp. 3537–3574.
- [6] R. Balzer, An 8-state minimal time solution to the firing squad synchronization problem, *Information and Control* 10 (1) (1967) 22–42.
- [7] H.-D. Gerken, Über Synchronisationsprobleme bei Zellularautomaten, Diplomarbeit, Institut für Theoretische Informatik, Technische Universität Braunschweig (1987).
- [8] J. Mazoyer, A six-state minimal time solution to the firing squad synchronization problem, *Theoretical Computer Science* 50 (2) (1987) 183–238.
- [9] P. Sanders, Massively parallel search for transition-tables of polyautomata, in: *Parcella '94: Proceedings of the VI International Workshop on Parallel Processing by Cellular Automata and Arrays*, Akademie Verlag, Berlin, 1994, pp. 99–108.
- [10] T. Correa, B. Gustavo, L. Lemos, A. Settle, An overview of recent solutions to and lower bounds for the firing synchronization problem, *arXiv:1701.01045* (2017).
- [11] M. Clergue, S. Verel, E. Formenti, An iterated local search to find many solutions of the 6-states firing squad synchronization problem, *Applied Soft Computing* 66 (2018) 449–461.
- [12] T. T. Nguyen, L. Maignan, Exploring millions of 6-state FSSP solutions: the formal notion of local CA simulation, in: *Cellular Automata and Discrete Complex Systems (AUTOMATA 2020)*, Vol. 12286 of *Lecture Notes in Computer Science*, Springer, 2020, pp. 1–13.

- [13] H. Umeo, T. Yanagihara, A smallest five-state solution to the firing squad synchronization problem, in: *Machines, Computations, and Universality (MCU 2007)*, Vol. 4664 of *Lecture Notes in Computer Science*, Springer, 2007, pp. 291–302.
- [14] J.-B. Yunès, A 4-states algebraic solution to linear cellular automata synchronization, *Information Processing Letters* 107 (2) (2008) 71–75.
- [15] H. Umeo, N. Kamikawa, J.-B. Yunès, A family of smallest symmetrical four-state firing squad synchronization protocols for ring arrays, *Parallel Processing Letters* 19 (2) (2009) 299–313.
- [16] J. Duprat, Firing squad synchronization problem: a formal proof of Mazoyer’s six-state solution in Coq, Coq user contribution **FiringSquad**, LIP, ENS Lyon; archived at <https://github.com/rocq-archive/firing-squad> (1997).
- [17] M. L. Minsky, *Computation: Finite and Infinite Machines*, Prentice-Hall, Englewood Cliffs, NJ, 1967.
- [18] J. Mazoyer, On optimal solutions to the firing squad synchronization problem, *Theoretical Computer Science* 168 (2) (1996) 367–404.
- [19] A. Settle, J. Simon, Smaller solutions for the firing squad, *Theoretical Computer Science* 276 (1–2) (2002) 83–109.
- [20] A. Berthiaume, T. Bittner, L. Perković, A. Settle, J. Simon, Bounding the firing synchronization problem on a ring, *Theoretical Computer Science* 320 (2–3) (2004) 213–228.
- [21] M. J. H. Heule, O. Kullmann, V. W. Marek, Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer, in: *Theory and Applications of Satisfiability Testing (SAT 2016)*, Vol. 9710 of *Lecture Notes in Computer Science*, Springer, 2016, pp. 228–245.
- [22] M. J. H. Heule, Schur number five, in: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18)*, 2018, pp. 6598–6606.
- [23] A. Biere, K. Fazekas, M. Fleury, M. Heisinger, CaDiCaL, Kissat, Para-cooba, Plingeling and Treengeling entering the SAT Competition 2020, in: *Proceedings of SAT Competition 2020: Solver and Benchmark Descriptions*, Vol. B-2020-1 of *Department of Computer Science Report Series B*, University of Helsinki, 2020, pp. 51–53.

- [24] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, F. Pollitt, CaDiCaL 2.0, in: Computer Aided Verification (CAV 2024), Vol. 14681 of Lecture Notes in Computer Science, Springer, 2024, pp. 133–152.
- [25] A. Ignatiev, A. Morgado, J. Marques-Silva, PySAT: a Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing (SAT 2018), Vol. 10929 of Lecture Notes in Computer Science, Springer, 2018, pp. 428–437.
- [26] N. Wetzler, M. J. H. Heule, J. Hunt, Warren A., DRAT-trim: efficient checking and trimming using expressive clausal proofs, in: Theory and Applications of Satisfiability Testing (SAT 2014), Vol. 8561 of Lecture Notes in Computer Science, Springer, 2014, pp. 422–429.
- [27] M. J. H. Heule, O. Kullmann, S. Wieringa, A. Biere, Cube and conquer: guiding CDCL SAT solvers by lookaheads, in: Hardware and Software: Verification and Testing (HVC 2011), Vol. 7261 of Lecture Notes in Computer Science, Springer, 2012, pp. 50–65.

Appendix A. The rule of Theorem 6.1

Appendix B. Reproducibility

The directory `fssp5/` of the accompanying repository contains all programs, the rule tables and the logs. The main components are:

`src/fsspcheck.c`

independent brute-force simulator: reads a rule table and checks minimal-time synchronization for a range of lengths (full simulation of every line, no use of the agreement lemma);

`src/gencnf.py`

CNF generator for $MT_k(\mathcal{N}; M, N')$ with the options of Section 4 (return chains, pumping, links, symmetry breaking) and the restricted classes used in the experiments;

`src/pumping.py`

computation of the cones of Lemma 3.2 by breadth-first search and direct verification of $PUMP(n, n')$ on a given rule;

`src/fsspdfs.c`

exhaustive depth-first enumeration of partial rules (Table 1);

left	centre	right neighbour				
		*	L	G	A	B
*	L	—	A	A	A	A
L	L	L	L	L	B	A
G	L	A	G	G	L	A
A	L	B	A	B	A	A
B	L	G	B	L	A	A
*	G	—	A	A	A	A
L	G	B	B	G	L	B
G	G	G	G	G	A	B
A	G	A	L	A	A	G
B	G	B	G	L	A	A
*	A	—	G	B	F	L
L	A	A	L	A	A	B
G	A	B	L	A	B	A
A	A	F	B	G	F	A
B	A	A	B	A	G	A
*	B	—	A	A	A	A
L	B	G	B	L	B	B
G	B	A	A	B	B	G
A	B	L	B	A	A	G
B	B	G	A	B	B	L

Table A.3: The five-state rule δ_{12} of Theorem 6.1: entry $\delta(\text{left}, \text{centre}, \text{right})$. The entries $\delta(L, L, L) = \delta(L, L, *) = L$ are the quiescence conditions; “—” marks the unused neighbourhoods $(*, y, *)$. Machine-readable copy: **results/uniformA_2-12.txt**.

`src/mazoyer_from_coq.py`
 extraction of Mazoyer's rule from the file `autom.v` of Duprat's Coq development [16];

`src/validate_mazoyer.py`
 soundness test of the encoding;

`src/check_halfline.py`
 direct check that a half-line diagram has no firing cell below a given anti-diagonal;

`src/fixgerm.py`, `src/knuth.py`, `src/benders.py`
 the experiments of Section 6.

Certificate for Theorem 5.1.

```
python3 gencnf.py 4 9 mt4_9.cnf
cadical --lrat --binary=false mt4_9.cnf mt4_9.lrat # UNSAT
lrat-check mt4_9.cnf mt4_9.lrat # VERIFIED
```

SHA-256 digests (each on two lines):

```
mt4_9.cnf          d18d65480e77505193b8f60362badad5
                   e71f8f08505f29befbcd3b2b18cdaabf
mt4_9.lrat         1d511a75ab25587fa4a3476b95ad3f71
                   33c7375ad96661fd22fa598aec6c8092
uniformA_2-12.txt  b62abbcac0653a661cd1bbe344dee9db
                   ba0cddee37448ea16681d436166c3aad
mazoyer6.txt       4b0717ea7319957c1fdcc887fe09c107
                   4cf338e8b849cdd32b4307d0083f417e
```

Checking Theorem 6.1.

```
R=results/uniformA_2-12.txt
fsspcheck $R 2 12      # OK for all 2 <= n <= 12
fsspcheck $R 13 13     # FAIL: a cell fires at time 23
python3 pumping.py $R 40 # 0 violations
python3 check_halfline.py $R 78 # no firing cell, t+i <= 78
```

The uniform configuration before firing in (a) is visible with the option `-v` of the checker, which prints the diagrams.

Validation against Mazoyer's solution.

```
python3 mazoyer_from_coq.py autom.v > mazoyer6.txt
fsspcheck mazoyer6.txt 3 3000 # OK for all 3 <= n <= 3000
python3 validate_mazoyer.py 14 # SAT under exactly one renaming
```